NU_Analytics at Daleel 2026: Independent Label Scoring and Structured Span Prediction for Arabic Argumentative Discourse Mining

Abdelrahman Ahmad*
AI Labs
Precision AI
Calgary, Canada
abdelrahman.ahmad@precision.ai

Hend Emad*
Computer Science
Nile University
Giza, Egypt
h.emad2487@nu.edu.eg

Ensaf Hussein
Artificial Intelligence
Nile University
Giza, Egypt
EnMohamed@nu.edu.eg

Abstract

We present our closed-track systems for both subtasks of Daleel 2026, which targets Arabic argumentative discourse unit (ADU) mining over editorials and competitive debates. Task 1 is an imbalanced multi label paragraph classification problem, and Task 2 requires character-level detection and classification of ADU spans.

For Task 1 we investigate two parameter-efficient decoder-LLM formulations: joint label generation with Qwen3.5-9B and independent label scoring with ALLaM-7B. The latter decomposes each paragraph into six explicit Yes/No decisions, giving every label direct supervision and a separate decision score; it achieves **0.6714** Macro-F1 on the evaluation phase and ranked **4th**, while ablations show that additional class-balanced weighting degrades performance.

For Task 2 we combine task-adaptive pretraining, Beginning-Inside-Outside Conditional Random Field (BIO-CRF) sequence labeling, boundary-jitter augmentation, character-space ensembling, and span post-processing over AraBERT-large, achieving an official F1 of **0.7523** (**0.7931** on debates, **0.6417** on editorials) and ranking **second**.

Across both tasks, our results support explicit label-wise supervision and span-level ensemble decisions as effective ways to handle label imbalance and boundary noise without aggressive reweighting, while short spans, rare classes, and genre variation remain challenging.

1 Introduction

Argumentative discourse unit (ADU) mining aims to identify argumentative content and its discourse roles. Daleel 2026, organized as part of ArabicNLP 2026 (ara, 2026), introduces this setting for Arabic across editorial and competitive-debate text (Nabhani et al., 2026b). The shared task presents the problem at two levels of detail.

Task 1 requires identifying all argumentative discourse unit types expressed in a paragraph, resulting in an imbalanced multi-label classification problem in which frequent labels such as Assumption (`AS`) coexist with much rarer categories such as Statistics (`ST`) and Common Ground (`CO`).

Task 2 requires systems to identify character-level ADU boundaries and assign one of the same six discourse labels to each detected span. We participate in the closed track of both tasks. The two granularities call for different modeling families, and we treat them as such.

For Task 1 we investigate two parameter-efficient decoder-LLM formulations: Qwen3.5-9B is adapted with LoRA to generate the complete label set as structured JSON, while ALLaM-7B uses QLoRA and decomposes prediction into six independent Yes/No label decisions. This second formulation gives every label explicit supervision and exposes a separate decision score for threshold selection.

For Task 2 we use a BIO-CRF architecture built on Arabic pretrained encoders, combining task-adaptive pretraining (TAPT), boundary-jitter augmentation, class-aware supervision, character-space ensembling, and span post-processing. The two systems share a single data split and a common lightweight discourse-cue detector, which each formulation consumes in the way its input representation allows. Independent label scoring achieves strong Macro-F1 without aggressive minority-class weighting, and the Task 1 submission ranked **4th**; for Task 2, larger encoders and multi-seed ensembling both help, and our four-seed AraBERT-large submission ranked **second**. Short spans and the editorial/debate dif-

*Equal contribution.

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 1358–1369
October 28-29, 2026 ©2026 Association for Computational Linguistics

ference remain challenging in both tasks. Code and configurations: https://github.com/Abdelrahman350/BoostedSpan.

2 Background

Daleel 2026 addresses Arabic argumentative discourse mining across *editorials* and *competitive debates* (Nabhani et al., 2026a). Both subtasks use the same six ADU types: Assumption (AS), Anecdote (AN), Statistics (ST), Testimony (TE), Common Ground (CO), and Other (OT). The released labeled data contain 612 paragraphs, which we use for both tasks. Task 1 requires predicting which of the six types occur in each paragraph; since multiple types may occur in the same paragraph, it is a multi-label classification problem. Task 2 requires detecting ADU character spans and assigning one of the six types to each, and is evaluated with a modified F1 measure that gives partial credit to overlapping predicted and gold spans. We participate in the closed track of both tasks.

Related work. Arabic-specific pretrained models such as AraBERT (Antoun et al., 2020) and ARBERT/MARBERT (Abdul-Mageed et al., 2021) established strong encoder-based approaches for Arabic text understanding and classification. More recent decoder-only LLMs extend this setting beyond task-specific classification heads. The Qwen family (Team, 2026) provides multilingual instruction-following models, while ALLaM (Bari et al., 2024) focuses specifically on Arabic-English language modeling. Argument mining more broadly has been widely studied in NLP (Lawrence and Reed, 2019), but Arabic remains comparatively under-resourced. Munazarat 1.0 provides Arabic competitive-debate data for argumentation research (Khader et al., 2024), while Nabhani and Al Khatib (2025) studied Arabic span-level argument mining using BIO labeling and cross-lingual transfer. Daleel provides native Arabic annotations across debate and editorial genres. Daleel Task 1 presents a different setting from standard single-label classification: several argumentative functions may occur in the same paragraph, and some labels have limited training support. We therefore study two decoder-LLM formulations for this setting, using Qwen3.5 for joint structured generation and ALLaM for independent label scoring and calibration. For Task 2, building on sequence-labeling approaches, we combine structured prediction with task adaptation, boundary augmentation, and character-space ensembling to address span boundaries and label imbalance.

Shared discourse cues. Both systems share one lightweight component. We use regex-based lexical cues for three labels with recognizable surface patterns: Testimony (TE), Statistics (ST), and Anecdote (AN). TE patterns capture reporting and attribution expressions, ST patterns capture numerical and statistical expressions, and AN patterns capture explicit personal-narrative expressions. The regular expressions are matched against the raw Arabic paragraph before any tokenization or prompt construction; the exact surface patterns covered by each cue family are listed in Appendix A. In both tasks, detected cues provide auxiliary lexical information and are never converted directly into predicted labels, so the model still makes the final decision from the complete paragraph. The two tasks differ only in how the cues enter the input. In Task 1, the detected cues are inserted into the ALLaM prompt as a natural-language hint. In Task 2, the detected label codes are inserted as a compact prefix such as `[CUES:TE,ST]` ahead of the paragraph, alongside a domain prefix, `[EDITORIAL]` or `[DEBATE]`.

3 Task 1: Multi-Label ADU Classification

We explored two formulations for adapting decoder LLMs to Task 1. Qwen3.5-9B predicts the complete label set through constrained generation, while ALLaM-7B scores each label independently through binary label scoring. Within the latter formulation, we further study class-imbalance handling and use label-specific decision thresholds. Figure 1 in Appendix K gives an overview of the Task 1 system.

3.1 Qwen3.5-9B: Constrained Generation

Our first system fine-tunes Qwen/Qwen3.5-9B with LoRA (Hu et al., 2022), formulating Task 1 as instruction-following generation rather than adding a classification head: the model emits the complete applicable label set as a compact JSON object of the form `{"labels":[...]}`. Imbalance is handled by oversampling the rare ST and CO classes rather than by class-weighted loss, and prediction is deterministic, so no threshold selection is used. Prompt formulation, output valida-

tion, and checkpoint selection are detailed in Appendix B.

3.2 ALLaM-7B: Independent Label Scoring

Our second system fine-tunes ALLaM-7B-Instruct-preview with QLoRA (Dettmers et al., 2023). The backbone is quantized to 4-bit NF4 and adapted using trainable LoRA parameters. Instead of predicting all labels jointly, we evaluate each label independently. Each paragraph is paired with each of the six ADU types, producing six binary training instances. For every paragraph–label pair, an Arabic instruction asks whether the paragraph contains the corresponding argumentative category.

During supervised fine-tuning, the target completion is the Arabic word for "Yes" for a positive pair and the Arabic word for "No" otherwise. The prompt tokens are masked from the loss, so supervision is applied only to the target answer.

Prompt formulation. The prompt specifies the paragraph domain, includes the original paragraph, provides an Arabic description of the candidate label, and requests a single Yes/No answer. Editorial paragraphs are described as newspaper editorials, while debate paragraphs are described as discussions or debates. When relevant discourse cues are detected (Section 2), they are inserted as a natural-language hint in the same prompt rather than used as prediction rules. The complete Arabic prompt template, label descriptions, discourse-cue descriptions, and exact Yes/No target completions are provided in Appendix D. At inference time, we do not generate the answer. Instead, we extract the next-token logits corresponding to the Arabic Yes and No answers. For label c, the decision score is

$$p_c = \frac{\exp(z^c_{\text{yes}})}{\exp(z^c_{\text{yes}}) + \exp(z^c_{\text{no}})}. \quad (1)$$

The normalized Yes/No logits provide a continuous score for each label. We do not assume that these scores are probabilistically calibrated; instead, they are treated as discriminative scores from which label-specific decision thresholds are selected on the validation set.

3.3 Imbalance and Threshold Selection

We study two complementary strategies for handling label imbalance in the ALLaM formulation: loss weighting during training and label-specific threshold selection at inference.

Imbalance handling. For the weighted variants, we assign each label c a positive-class weight based on its frequency in the training split:

$$w_c = \min\left(C, \frac{N - N^+_c}{N^+_c}\right),$$

where N is the number of training examples, N^+_c is the number of positive examples for label c, and C controls the maximum weight. The weight is applied only to positive instances of the corresponding label, increasing the loss contribution of rare positive examples. We evaluate two settings, $C = 8$ and a more conservative $C = 3$, together with an unweighted baseline.

Threshold selection. Checkpoint selection and threshold tuning are performed separately. During training, checkpoints are evaluated using Macro-F1 with a fixed threshold of 0.5, and the checkpoint with the highest validation score is retained. After model selection, a separate threshold is tuned for each label. We sweep thresholds from 0.05 to 0.95 in increments of 0.05 and select the value that maximizes the corresponding label's F1 score on the validation set. These thresholds are then fixed for final prediction.

4 Task 2: Character-Level Span Detection

We formulate Task 2 as BIO sequence labeling followed by character-level span reconstruction. Figure 2 in Appendix L summarizes the two stages of our pipeline: model development and inference. The submitted system uses AraBERT-large (Antoun et al., 2020) (`aubmindlab/bert-large-arabertv02`) with four independently trained runs.

The input representation and label-alignment procedure are detailed in Appendix E.

4.1 Training and Model Development

We first apply task-adaptive pretraining (TAPT) (Gururangan et al., 2020) by continuing masked-language-model training of AraBERT-large on the Daleel training and `dev_in` text without ADU labels. The adapted encoder is then fine-tuned for supervised span detection. Gold character spans are mapped to 13 BIO tags: `O`, `B-c`, and `I-c` for the six ADU classes, using tokenizer offset mappings (Ramshaw and Marcus, 1995). Before tokenization, the discourse-cue and domain prefixes

described in Section 2 are prepended to the paragraph.

Boundary-jitter augmentation. To reduce sensitivity to small annotation-boundary variations, we create additional training copies in which each gold span is perturbed independently. Its start and end offsets are each shifted by an integer sampled uniformly from $[-2, 2]$ characters. The shifted start is clipped to the valid text range, while the shifted end is clipped to the paragraph boundary and constrained to remain at least one character after the new start. BIO labels are then reconstructed from the perturbed spans. Because a single BIO sequence cannot represent overlapping spans, overlaps are resolved using a first-writer-wins policy: character positions already assigned to an earlier span are not overwritten by subsequent spans.

Structured prediction and weighted supervision. AraBERT representations are projected into BIO emission scores and decoded with a linear-chain CRF (Lafferty et al., 2001). The standard objective minimizes CRF negative log-likelihood. To address label imbalance, we additionally evaluate

$$\mathcal{L} = \mathcal{L}_{\text{CRF}} + \lambda \mathcal{L}_{\text{WCE}}, \tag{2}$$

where the auxiliary weighted cross-entropy assigns greater importance to rare BIO tags. Four AraBERT-large models are trained with different random seeds for the final ensemble.

4.2 Inference and Span Reconstruction

At inference, paragraphs are tokenized with character offsets and discourse-cue markers, CRF emissions are decoded with Viterbi, and BIO predictions are mapped back to character coordinates so runs can be combined in a common character space. For character j and class c, support is

$$V(j, c) = \sum_{m=1}^{M} w_m \, \mathbb{I}\left[j \in S_c^{(m)}\right] \tag{3}$$

where w_m is based on validation performance. We also evaluate span-cluster voting, which groups overlapping predicted spans before voting. Long-input handling, the isolated-`I` rule, and the span post-processing filters are in Appendix F.

5 Experimental Setup

Both tasks draw on the same 612 labeled Daleel paragraphs. The ALLaM Task 1 experiments and all Task 2 experiments use one fixed split of approximately 520 training and 92 internal-validation paragraphs; Qwen3.5 instead trains on all 612 and validates on the 217 development paragraphs, so the two Task 1 systems' validation scores are not a controlled comparison. Checkpoints are selected by validation Macro-F1 and per-label thresholds are tuned afterwards. Task 1 is scored with Micro-, Macro- and Weighted-F1; Task 2 with the official partial-overlap metric via the organizers' unmodified `task2_scoring.py` on the 92-paragraph split. All training runs on a single NVIDIA RTX 4080 12 GB GPU. Split construction, closed-track compliance, TAPT data, libraries and the full selection protocol are in Appendix G, and the complete optimization settings for both tasks in Appendix H.

Model	Domain	Micro	Macro	Weighted
Qwen3.5	All	0.7721	0.6215	0.7622
	Editorial	0.7730	0.5962	0.7504
	Debate	0.7715	0.4820	0.7622
ALLaM	All	0.7785	0.6714	0.7833
	Editorial	0.6493	0.4751	0.6252
	Debate	0.8460	0.7418	0.8514

Table 1: Task 1 phase-wise F1 scores. Qwen3.5 results are from the development phase and ALLaM results from the evaluation phase.

6 Results

6.1 Task 1 Results

Table 1 summarizes the available phase results for Task 1. Qwen3.5 is reported on the development phase, whereas ALLaM is reported on the evaluation phase; therefore, the scores are not treated as a controlled head-to-head comparison. Qwen3.5 obtained 0.6215 Macro-F1, with a larger degradation on debates (0.4820) than editorials (0.5962), despite similar Micro-F1 across genres. ALLaM reached 0.6714 Macro-F1 overall, with substantially stronger performance on debates (0.7418) than editorials (0.4751). Its strongest evaluation-phase classes were `AS` and `ST`, while `CO` remained the most difficult. Because both the backbone and evaluation phase differ, the numerical gap between Qwen3.5 and ALLaM should not be interpreted as a direct model comparison.

6.2 Task 2 Results

Table 2 reports the official closed-track results of our submitted Task 2 system. The system achieved an overall F1 of 0.7523, with 0.7931

Test subset	Precision	Recall	F1
Editorials	.6394	.6440	.6417
Debates	.7862	.8000	.7931
Overall	.7479	.7569	**.7523**

Table 2: Official Task 2 closed-track test results.

Configuration	Micro	Macro	Weighted
Base QLoRA	**0.8650**	**0.8337**	**0.8675**
Low-clip weighting	0.8538	0.8141	0.8558
Weighted loss	0.8507	0.7931	0.8515

Table 3: Task 1 ALLaM ablation results on the fixed validation split.

on debates and 0.6417 on editorials, ranking second on the Task 2 leaderboard. Recall is slightly higher than precision overall (0.7569 vs. 0.7479), with the same pattern in both genres, indicating no strong precision–recall imbalance. In contrast, the 0.1514 F1 gap between debates and editorials indicates genre variation. Since the test labels are hidden, we do not attribute this difference to a specific error type; instead, the following analyses use the held-out validation split.

6.3 Ablation Studies

Task 1: imbalance handling. Table 3 compares the ALLaM variants on the same fixed 92-paragraph validation split using the organizers' scorer. The unweighted system performs best. Class-balanced weighting with a maximum weight of 8 reduces Macro-F1 from 0.8337 to 0.7931; lowering the cap to 3 partially recovers performance to 0.8141. This suggests that aggressively upweighting the few positive instances of rare labels can give individual examples disproportionate influence.

Task 2: design choices. Table 4 reports the main Task 2 design choices on the same fixed 92-paragraph validation split using the official partial-overlap scorer. These are internal validation results and are separate from the official leaderboard score. Adding class-weighted cross-entropy increased F1 from 0.6967 to 0.7059. The gain mainly came from precision, while recall remained almost unchanged. This suggests that additional supervision for rare BIO tags helped reduce incorrect span predictions without reducing span coverage. Changing the ensemble decoder had a larger effect. Span-cluster decoding increased F1 from 0.7059 to 0.7294 and raised precision from 0.6651 to 0.7317. This supports our motivation for moving the ensemble decision from individual characters to groups of overlapping spans. Increasing the AraBERT-large ensemble from two to four independent runs improved validation F1 from 0.7352 to 0.7438. Further analysis of encoder capacity and ensemble size is provided in Appendix I.

Configuration	Precision	Recall	F1
Base Track A	0.6494	0.7514	0.6967
weighted CE	0.6651	0.7521	0.7059
span-cluster decoding	0.7317	0.7271	0.7294
AraBERT-large, 2 seeds	0.7120	0.7601	0.7352
AraBERT-large, 4 seeds	0.7308	0.7572	**0.7438**

Table 4: Task 2 validation results for the main design choices. The large-backbone experiments are controlled variants rather than cumulative additions to the clustered system.

6.4 Error Analysis

The main limitations are rare ADU labels, genre variation, short-span loss, and overlapping annotations. Detailed analyses are provided in Appendix J.

7 Conclusion

We presented closed-track systems for both subtasks of Daleel 2026. For Task 1 we compared two parameter-efficient decoder-LLM formulations, joint label generation with Qwen3.5-9B and independent label scoring with ALLaM-7B. Explicit label-wise supervision suits the imbalanced multi-label setting, while additional class-balanced weighting does not help; the ALLaM submission achieved **0.6714** Macro-F1 and ranked 4th. For Task 2, a BIO-CRF system combining task-adaptive pretraining, boundary augmentation, AraBERT-large and character-space ensembling ranked second with an official F1 of **0.7523**; larger encoders, multi-seed ensembling and span-cluster decoding each helped on validation. Across both tasks, short spans, rare ADU types such as `CO`, overlapping units, and cross-genre variation remain challenging. Future work should investigate label-specific threshold tuning, stronger methods for rare labels, and domain-aware training. A controlled comparison of the Task 1 formulations using the same backbone and data split, alongside span-based architectures for Task 2, would help address these limitations.

References

2026. The fourth arabic natural language processing conference (arabicnlp 2026). https://arabicnlp2026.sigarab.org/. Co-located with EMNLP 2026, Budapest, Hungary, October 24--29, 2026.

Muhammad Abdul-Mageed, AbdelRahim Elmadany, and El Moatez Billah Nagoudi. 2021. ARBERT & MARBERT: Deep bidirectional transformers for Arabic. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 7088--7105, Online. Association for Computational Linguistics.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2020. AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools, with a Shared Task on Offensive Language Detection*, pages 9--15, Marseille, France. European Language Resources Association.

M. Saiful Bari, Yazeed Alnumay, Norah A. Alzahrani, Nouf M. Alotaibi, Hisham A. Alyahya, Sultan AlRashed, Faisal A. Mirza, Shaykhah Z. Alsubaie, Hassan A. Alahmed, Ghadah Alabduljabbar, Raghad Alkhathran, Yousef Almushayqih, Raneem Alnajim, Salman Alsubaihi, Maryam Al Mansour, Majed Alrubaian, Ali Alammari, Zaki Alawami, Abdulmohsen Al-Thubaity, and 6 others. 2024. ALLaM: Large language models for arabic and english. *arXiv preprint arXiv:2407.15390*.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems*, volume 36.

Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. 2020. Don't stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8342--8360. Association for Computational Linguistics.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Mohammad M. Khader, AbdulGabbar Al-Sharafi, Mohamad Hamza Al-Sioufy, Wajdi Zaghouani, and Ali Al-Zawqari. 2024. Munazarat 1.0: A corpus of Arabic competitive debates. In *Proceedings of the 6th Workshop on Open-Source Arabic Corpora and Processing Tools (OSACT) with Shared Tasks on Arabic LLMs Hallucination and Dialect to MSA Machine Translation @ LREC-COLING 2024*, pages 20--30, Torino, Italia. ELRA and ICCL.

John Lafferty, Andrew McCallum, and Fernando C. N. Pereira. 2001. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. In *Proceedings of the Eighteenth International Conference on Machine Learning*, pages 282--289. Morgan Kaufmann Publishers Inc.

John Lawrence and Chris Reed. 2019. Argument mining: A survey. *Computational Linguistics*, 45(4):765--818.

Sara Nabhani and Khalid Al Khatib. 2025. Transfer or translate? argument mining in Arabic with no native annotations. In *Proceedings of The Third Arabic Natural Language Processing Conference*, pages 407--416, Suzhou, China. Association for Computational Linguistics.

Sara Nabhani, Nahla Bassyouni, Ali Al-Zawqari, Mohammad Khader, and Khalid Al-Khatib. 2026a. Daleel 2026: Arabic argumentative discourse mining shared task. ArabicNLP 2026 Shared Task. Official shared-task website.

Sara Nabhani, Nahla Bassyouni, Ali Al-Zawqari, Mohammad Khader, and Khalid Al-Khatib. 2026b. Daleel 2026 shared task on arabic argumentative discourse mining. Budapest, Hungary. Association for Computational Linguistics.

Lance A. Ramshaw and Mitchell P. Marcus. 1995. Text chunking using transformation-based learning. In *Proceedings of the Third Workshop on Very Large Corpora*, pages 82--94.

Qwen Team. 2026. Qwen3.5-omni technical report. *arXiv preprint arXiv:2604.15804*.

A Discourse-Cue Patterns

The shared discourse-cue detector described in Section 2 uses regular expressions for three labels with recognizable surface patterns.

- **TE** patterns capture reporting and attribution expressions, including equivalents of "said," "stated," "confirmed," "according to," and "mentioned that."
- **ST** patterns capture numerical and statistical expressions, including percentages and terms referring to studies, statistics, scientific research, and ratios.
- **AN** patterns capture explicit personal-narrative expressions, including equivalents of "I remember," "it happened to me," "once," "my story," and "my personal experience."

The regular expressions are matched against the raw Arabic paragraph before prompt construction (Task 1) or tokenization (Task 2). If one or more cues are detected, their semantic descriptions are added to the ALLaM prompt as an auxiliary natural-language hint (Task 1) or their label codes are inserted as a compact prefix such as `[CUES:TE,ST]` (Task 2). They are never converted directly into predicted labels.

B Qwen3.5 System Details

Our first system fine-tunes Qwen/Qwen3.5-9B with LoRA (Hu et al., 2022). Rather than adding a classification head, we formulate Task 1 as instruction-following generation. The model receives a system instruction defining the six ADU labels and the required output format, followed by a user message containing the paragraph domain and text. It generates the complete applicable label set as a compact JSON object of the form `{"labels":[...]}`.

Prompt formulation. The system prompt defines AS, AN, ST, TE, CO, and OT in Arabic and instructs the model to return valid JSON using only the `labels` key, without additional explanation. It also specifies the fixed output order AS, AN, ST, TE, CO, OT and permits an empty list when no category applies. The user message inserts `{DOMAIN}` and `{TEXT}` into an Arabic instruction of the form "text type: {DOMAIN}; paragraph: {TEXT}; classify the paragraph and return JSON only." The exact Arabic system and user prompts are provided in Appendix C. During supervised fine-tuning, the target is the gold label set serialized in the fixed order, for example `{"labels":["AS","TE"]}`. Prompt tokens are masked from the language-modeling loss, so supervision is applied only to the generated JSON response.

Imbalance handling. Unlike the ALLaM experiments, the Qwen system does not use class-weighted loss. Instead, we increase the exposure of the rare ST and CO classes through training-set oversampling. Every paragraph containing at least one of these labels is duplicated once, corresponding to a repeat factor of two, while other paragraphs appear once.

Prediction and model selection. Qwen predicts labels directly through deterministic generation (`do_sample=False`) rather than independent label scores. Consequently, no probability threshold or label-specific threshold selection is used. Generated responses are parsed as JSON and checked for the required `labels` key, valid label names, and duplicate labels; valid labels are then normalized to the fixed label order. The model is evaluated after every epoch, and the checkpoint with the highest validation Macro-F1 is retained.

C Qwen3.5 Prompt Specification

Qwen3.5 is trained in a chat-style instruction-following setup. Each training example contains a fixed Arabic system prompt, a user message constructed from the paragraph domain and text, and an assistant target containing the gold label set in JSON format.

System prompt. The exact system prompt used in our implementation is:

أنت نظام متخصص في تصنيف وحدات الخطاب الحجاجي العربي.
المهمة متعددة التصنيفات؛ قد تنطبق تسمية واحدة أو عدة تسميات على الفقرة.
التسميات المسموح بها فقط:
AS: موقف أو ادعاء أو حجة أساسية يعرضها الكاتب أو المتحدث.
AN: دليل أو مثال أو تفسير غير إحصائي يدعم موقفًا أو حجة.
ST: دليل رقمي أو إحصائي أو نسبة أو كمية تستخدم كدليل.
TE: نقل أو تلخيص أو مناقشة أو رد على كلام طرف أو متحدث آخر.

CO: مقارنة أو تعارض مباشر بين موقفين أو خيارين أو حالتين.
OT: محتوى تنظيمي أو تمهيدي أو ختامي أو خارج الوظائف السابقة.
قواعد الإجابة:
1. أعد صحيحًا فقط دون شرح. JSON
2. استخدم المفتاح labels فقط.
3. لا تستخدم أي تسمية خارج القائمة.
4. رتب التسميات هكذا: AS, AN, ST, TE, CO, OT.
5. استخدم قائمة فارغة فقط إذا لم تنطبق أي تسمية.
الشكل المطلوب:
`{"labels":["AS","TE"]}`

User prompt. For each paragraph, the user message is constructed as:

صنّف الفقرة {TEXT} الفقرة: {DOMAIN} نوع النص:
فقط. JSON وأعد

Here, {DOMAIN} is the paragraph type (editorial or debate), and {TEXT} is the original Arabic paragraph.

Target output. During supervised fine-tuning, the assistant target is the gold set of labels serialized as compact JSON using the fixed order AS, AN, ST, TE, CO, OT. For example:

`{"labels":["AS","TE"]}`

If no label applies, the target is:

`{"labels":[]}`

Prompt tokens are masked from the language-modeling loss, so the loss is computed only over the assistant JSON response. At inference time, generation is deterministic, with sampling disabled, and the generated text is parsed to recover the predicted label set.

D ALLaM Prompt Specification

For each paragraph, ALLaM is queried independently for each of the six ADU labels. The prompt contains the paragraph domain, the original Arabic paragraph, an Arabic description of the candidate label, and an optional discourse-cue hint. The exact prompt template used in our implementation is shown below.

فيما يلي فقرة من {DOMAIN}: {TEXT}
هل تحتوي هذه الفقرة على عنصر حجاجي من النوع التالي:
{LABEL_DESCRIPTION}؟
أجب بكلمة واحدة فقط: نعم أو لا. الإجابة:

The placeholders are instantiated as follows:

- {DOMAIN}: the paragraph domain, instantiated as افتتاحية صحفية for editorials and نقاش أو مناظرة for debates.
- {TEXT}: the original Arabic paragraph provided to the model.
- {LABEL_DESCRIPTION}: the Arabic natural-language description of the candidate ADU label currently being evaluated. Since each paragraph is evaluated against all six labels, this placeholder changes across the six paragraph–label instances.
- {CUE_NOTE}: an optional natural-language hint derived from detected discourse cues. It is empty when no cue is detected; otherwise, it contains one or more cue descriptions for TE, ST, and/or AN.

The six label descriptions are:

- **AS**: افتراض أو رأي شخصي غير مدعوم بدليل مباشر
- **AN**: قصة شخصية أو حكاية أو تجربة فردية
- **ST**: إحصائية أو بيانات رقمية أو نسبة مئوية
- **TE**: شهادة أو رأي منقول عن شخص أو مصدر آخر
- **CO**: أرضية مشتركة أو حقيقة متفق عليها بين الطرفين
- **OT**: نوع آخر من الحجج لا يندرج تحت الأنواع السابقة

When discourse cues are detected, {CUE_NOTE} is inserted after the label description using the following form:

(تحتوي الفقرة على مؤشرات لغوية قد تدل على:
{DESCRIPTIONS})

The cue descriptions are:

- **TE**: نقل كلام عن مصدر آخر
- **ST**: أرقام أو إحصاءات
- **AN**: سرد قصة أو تجربة شخصية

If multiple cues are detected, their descriptions are concatenated in the same hint. During supervised fine-tuning, the exact target completion is نعم for a positive paragraph–label pair and لا for a negative pair.
In the implementation, each target completion is preceded by a single whitespace character. At inference time, no answer is generated; instead, the next-token logits corresponding to these two Arabic answers are used to compute the label-specific decision score.

E Task 2: Input Representation and Label Alignment

Unlike Task 1, Task 2 is formulated as sequence labeling rather than generative prompting. Each input consists of the original Arabic paragraph preceded by a domain marker and, when applicable, a compact discourse-cue marker. The domain marker is `[EDITORIAL]` or `[DEBATE]`, while the cue marker contains the detected label codes, such as `[CUES:TE,ST]`. An illustrative input is:

```
[EDITORIAL] [CUES:TE,ST] <Arabic
paragraph>
```

The markers provide auxiliary domain and lexical information and are not converted directly into predicted ADU labels. The input is tokenized with character-offset mappings, and gold character spans are aligned with the corresponding paragraph tokens to construct the 13 BIO tags. At inference, predicted BIO sequences are mapped back to character coordinates to produce the required span-level output.

F Inference and Span Reconstruction Details

Approximately 99% of paragraphs fit within the 512-subword limit; inputs in the small long-input tail are processed using overlapping windows with stride 64. When CRF emissions are decoded with Viterbi, an incompatible isolated `I` tag is treated as the beginning of a new span. Mapping BIO predictions back to character coordinates matches the character-level output required by the task, and consecutive selected characters are reconstructed into spans. Span-cluster voting groups overlapping predicted spans before voting, which reduces the disagreement caused by small boundary shifts. Finally, we remove punctuation- or whitespace-only spans, expand boundaries falling inside words to word edges, and merge same-label spans separated by at most two punctuation or whitespace characters.

G Extended Experimental Setup

G.1 Data Splits

Both tasks draw on the same 612 labeled Daleel paragraphs. The ALLaM Task 1 experiments and all Task 2 experiments use the same fixed split of these paragraphs, with approximately 520 examples for training and 92 for internal validation; for Task 1 the split is stratified by domain and label combination and is kept fixed across the three ALLaM configurations, and for Task 2 the validation set is used for model selection, ensemble weighting, and ablation analysis. For Qwen3.5, we instead use all 612 labeled training paragraphs for fine-tuning and the 217 labeled development paragraphs for validation, with oversampling applied only to the training set. Since Qwen and ALLaM therefore use different validation sets, their internal validation scores are reported separately rather than treated as a controlled comparison.

For task-adaptive pretraining in Task 2, we use the raw text from the training data together with the provided `dev_in` text, without using any development or test labels. All submitted systems follow the closed-track restrictions.

G.2 Training Configuration

The full optimization settings for both tasks are reported in Appendix H; we summarize the choices that matter for interpreting the results here. All ALLaM variants use the same QLoRA configuration and differ only in their loss weighting.

For Qwen3.5, we use AdamW with a linear learning-rate schedule and gradient clipping at 1.0. The batch size of 1 with eight gradient accumulation steps gives an effective batch size of 8. Training uses bfloat16 when supported and float16 otherwise. Paragraphs containing `ST` or `CO` are duplicated once before training.

For ALLaM, the backbone is quantized using 4-bit NF4 with double quantization and bfloat16 computation. LoRA is applied to the attention projections (`q`, `k`, `v`, and `o`) and the MLP projections (`gate`, `up`, and `down`). A batch size of 2 with 32 accumulation steps gives an effective batch size of 64. Gradient checkpointing is enabled for both systems. The base ALLaM configuration uses the standard unweighted SFT objective. The weighted variant applies class-balanced per-example loss weighting with a maximum weight of 8, while the low-clip variant reduces this maximum to 3. All other training settings are kept unchanged across the three variants.

For Task 2, our final system uses `aubmindlab/bert-large-arabertv02` as the encoder. We first apply task-adaptive masked-language-model pretraining and then fine-tune the resulting checkpoint with the BIO-CRF span detector described in Section 4. The

final configuration consists of four independently trained AraBERT-large models with different random seeds. Inputs are limited to 512 subword tokens; longer paragraphs are processed using overlapping windows with a stride of 64 tokens. Boundary-jitter augmentation shifts span boundaries by at most two characters during training. For the weighted-loss ablation, the CRF objective is augmented with class-weighted token-level cross-entropy. We also compare character-level voting with span-cluster decoding, base- and large-capacity encoders, and different ensemble sizes. Unless otherwise stated, all other preprocessing and post-processing settings are kept fixed across comparisons.

G.3 Model Selection and Calibration

For Qwen3.5, we evaluate the model after each epoch and retain the checkpoint with the highest validation Macro-F1. Validation predictions are generated greedily with at most 48 new tokens. ALLaM checkpoints are also selected by validation Macro-F1, using a fixed threshold of 0.5 during checkpoint selection. After training, a separate decision threshold is selected for each label to maximize its F1 score on the validation set.

G.4 Evaluation

For Task 1, we report Micro-, Macro-, and Weighted-F1 for the development experiments and use Macro-F1 for model selection. Per-label precision, recall, and F1 are used for further analysis. For Task 2, we evaluate span predictions using the official Daleel Task 2 partial-overlap precision, recall, and F1 metric; all Task 2 validation results reported in Section 6 are computed by running the organizers' unmodified `task2_scoring.py` script on the 92-paragraph validation split. Official shared-task results for both tasks are reported separately in Section 6 and are not used for model selection.

Our Task 1 implementation uses PyTorch, Hugging Face Transformers, PEFT, scikit-learn, and bitsandbytes for the ALLaM QLoRA experiments. Task 2 models are implemented in PyTorch using Hugging Face Transformers and `torchcrf`. All training for both tasks is performed on a single NVIDIA RTX 4080 12 GB GPU.

H Training Configurations

Table 5 reports the main training configurations for the two Task 1 systems, and Table 6 reports the main experimental settings for the Task 2 system.

Setting	Qwen3.5	ALLaM
Backbone	Qwen3.5-9B	ALLaM-7B-Instruct
Adaptation	LoRA	QLoRA
Quantization	None	4-bit NF4
LoRA rank	16	16
LoRA α	32	32
LoRA dropout	0.05	0.05
Max. length	512	1024
Epochs	10	3
Batch size	1	2
Grad. accumulation	8	32
Learning rate	1×10^{-4}	2×10^{-4}
Weight decay	0.01	0.01
Warmup ratio	0.10	0.10
Seed	42	42

Table 5: Main training configurations for Task 1.

Setting	Value
Final backbone	AraBERT-large
Max. sequence length	512
Sliding-window stride	64
Boundary jitter	± 2 characters
BIO tags	13
Boosted-CRF training epochs	10
Boosted-CRF batch size	4/device
Boosted-CRF learning rate	3e-5
Boosted-CRF seeds	[42, 123]
Weight decay	0.01
Final ensemble	4 seeds
Ensemble weighting	Validation F1
TAPT objective	MLM
MLM probability	0.15
TAPT epochs	10
TAPT learning rate	5e-5

Table 6: Main experimental settings for Task 2.

I Additional Design-Choice Analysis

Increasing encoder capacity produced another improvement. Replacing the base-size encoders with AraBERT-large reached 0.7352 F1 with two seeds. Increasing the AraBERT-large ensemble from two to four independent runs further improved F1 to 0.7438, mainly through higher precision. This was the strongest validation configuration and was selected for the final submission. Overall, the experiments indicate that performance gains come from complementary sources: weighted supervision improves handling of imbalanced tags, span-level voting reduces boundary disagreement, and larger multi-seed ensembles improve prediction stability.

J Extended Error Analysis

Additional main-text error analysis. For Task 1, `CO` is the hardest class for both systems and ALLaM shows a pronounced genre effect

(0.7418 Macro-F1 on debates versus 0.4751 on editorials). For Task 2, the dominant error source is short-span loss during character-level ensembling, which motivated the span-cluster decoder, alongside severe span-level label imbalance and the inability of a single BIO sequence to represent the roughly 1% of paragraphs with overlapping gold spans. Per-label error profiles, span-length statistics, and a comparison of how the two Task 1 formulations differ in supervision are in Appendix J.

Task 1. Qwen3.5's main weakness is rare-label recall: `CO` achieves only 0.1538 F1 on validation despite perfect precision, while all generated outputs remain valid JSON. ALLaM shows a different error profile on the evaluation phase: `AN` is over-predicted ($P = 0.4500$, $R = 0.8000$), whereas `TE` is conservative ($P = 0.9211$, $R = 0.5000$). `CO` remains difficult for both systems. ALLaM also exhibits a pronounced genre effect, with 0.7418 Macro-F1 on debates versus 0.4751 on editorials.

Task 2. Our development analysis identified short-span loss during character-level ensembling as an important error source. Character voting removed approximately 26% of gold spans that had been detected by the individual models. These missed spans were considerably shorter on average; about 50 characters compared with about 90 characters for successfully matched spans, while the individual runs recalled 96--99% of them. Small differences in predicted boundaries distributed the ensemble votes across nearby characters, which motivated the span-cluster decoder. A second difficulty is label imbalance: the training data contain 1,739 AS spans but only 50 CO and 38 ST spans, and the improvement obtained from weighted supervision suggests that additional emphasis on rare BIO tags helps mitigate this imbalance. Finally, BIO tagging has a structural limitation: approximately 1% of the paragraphs contain overlapping gold spans, which cannot be represented by a single BIO sequence. These cases remain a limitation of the submitted system and motivate span-based architectures that can predict overlapping units directly.

Effect of the Task 1 prediction formulation. Beyond the backbone difference, the two Task 1 systems provide fundamentally different supervision. Qwen3.5 learns one jointly generated label sequence per paragraph, whereas ALLaM creates six paragraph–label instances with explicit Yes/No targets, giving each label a direct training signal for every paragraph. This is particularly useful for imbalanced labels, as reflected in the conservative behavior of Qwen3.5 on `CO`, which achieves 1.00 precision but only 0.0833 recall on validation. ALLaM further provides a separate normalized Yes/No score for each label, enabling label-specific threshold selection and more flexible control of the precision–recall trade-off. Together, these properties make independent label scoring a natural fit for the imbalanced multi-label setting. In our experiments, the ALLaM-based formulation achieves strong performance without class-balanced weighting, while the weighting ablations show that aggressively increasing the contribution of minority labels can instead hurt performance. Although the comparison involves different backbones and training setups, the results highlight independent label scoring as a promising formulation for addressing label imbalance in Arabic argument classification.

K Task 1 System Diagram

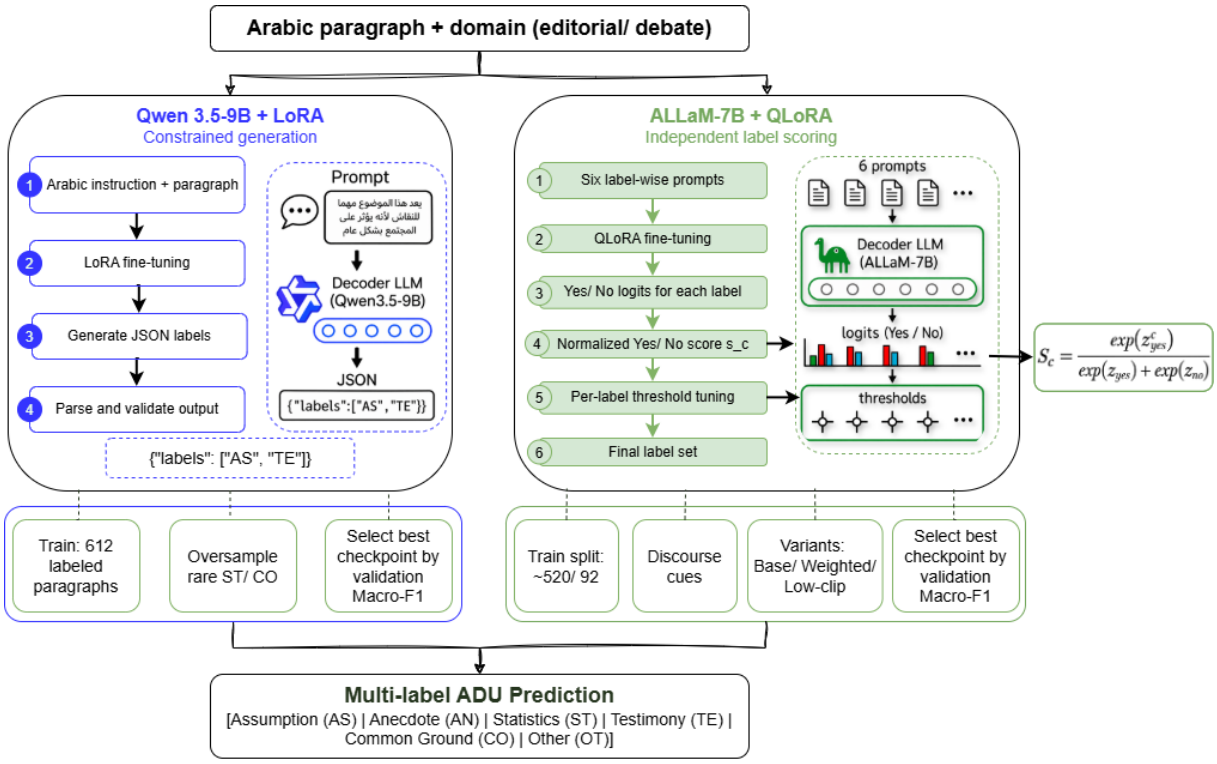


Figure 1: Overview of the Daleel Task 1 system.

L Task 2 System Diagram

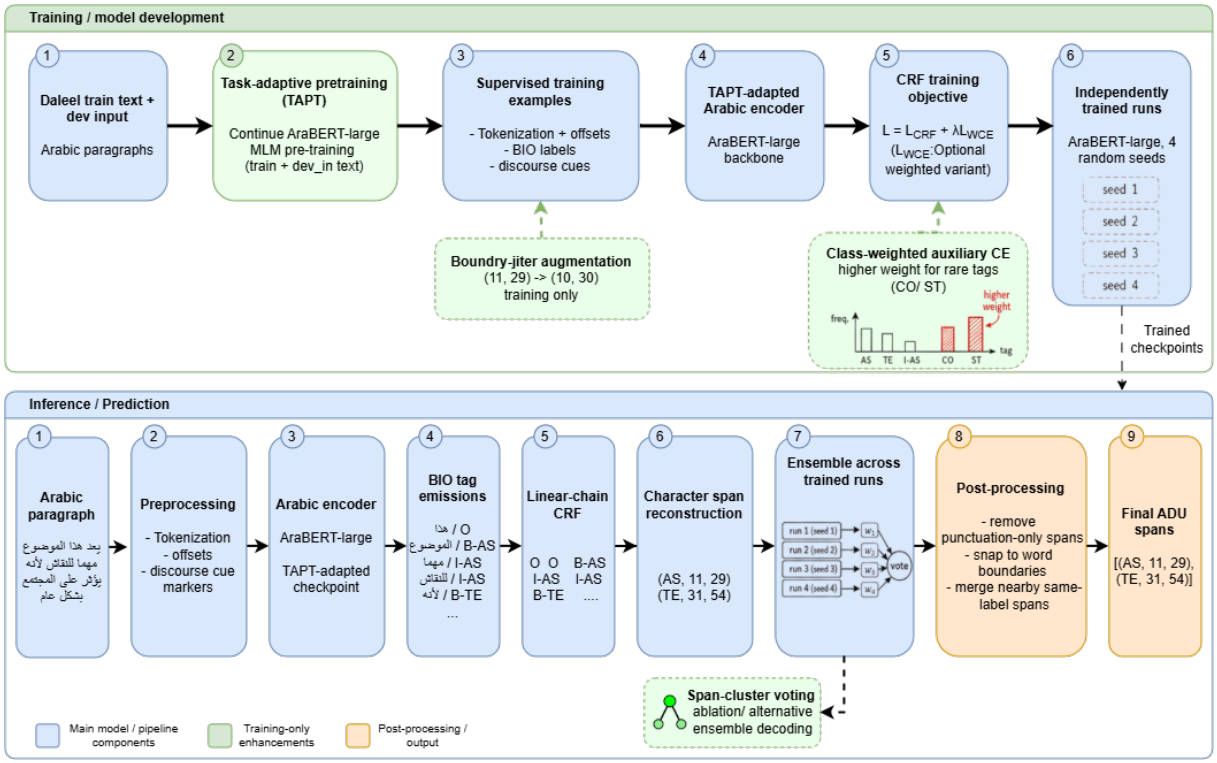


Figure 2: Overview of the Daleel Task 2 system.